

CSE 207 — Computational Complexity

Complete Exam Preparation Guide (*P*, *NP*, *co-NP*, *Reductions*, *NP-Completeness*)

Prepared for Rupom — based on Dr. Md. Shamsuzzoha Bayzid's lecture slides

Exam Prep — Night Before

How to Use This Guide

Table of Contents (Topic Map)

Topic 1: The Big Picture — Why Complexity Theory Exists

- 1.1 Simple Explanation
- 1.2 Why This Matters — The Core Puzzle
- 1.3 The Strategy Complexity Theory Uses
- 1.4 Slide-Based Q&A
- 1.5 Likely Exam Questions
- 1.6 Common Mistakes

Topic 2: Decision Problems vs. Optimization Problems

- 2.1 Simple Explanation
- 2.2 Converting Optimization $\rightarrow$ Decision
- 2.3 Why It Matters — Optimization $\neq$ Decision Are Equally Hard
- 2.4 Likely Exam Questions
- 2.5 Common Mistakes

Topic 3: The Classes *P* and *NP*

- 3.1 Formal Definitions
- 3.2 Simple, Intuitive Explanation
- 3.3 $P \subseteq NP$ (Every Problem in *P* Is Also in *NP*)
- 3.4 Does $NP \subseteq P$? (Is $P = NP$?)
- 3.5 Slide-Based Q&A
- 3.6 Likely Exam Questions
- 3.7 Common Mistakes / Traps
- 3.8 Practice (True/False with reasoning)

Topic 4: Complement of a Problem and *co-NP*

- 4.1 Simple Explanation
- 4.2 Complement and *P*: $x \in P \rightarrow x' \in P$
- 4.3 Does the Same Trick Work for *NP*? — No!
- 4.4 Definition of *co-NP*
- 4.5 Worked Example: PRIME
- 4.6 $P \subseteq \text{co-NP}$
- 4.7 Does $NP = \text{co-NP}$?
- 4.8 The Relationship Between *P*, *NP*, *co-NP* (Key Implication Chain)
- 4.9 Slide-Based Q&A
- 4.10 Likely Exam Questions
- 4.11 Common Mistakes

Topic 5: Polynomial-Time Reductions

- 5.1 Formal Definition and Intuitive Meaning
- 5.2 Why Reductions Matter: Difficulty and Efficiency Flow in Opposite Directions
- 5.3 The Warning Slide — Direction Matters!
- 5.4 Slide-Based Q&A
- 5.5 Likely Exam Questions
- 5.6 Common Mistakes

Topic 6: *NP*-hard and *NP*-complete Problems

- 6.1 *NP*-hard — Formal Definition
- 6.2 *NP*-complete — Formal Definition
- 6.3 The Venn Diagram Picture (“What we think the world looks like”)
- 6.4 Why “*NP*-hard” Is Not the Same as “*NP*-complete”
- 6.5 Proving *NP*-completeness — General Method

- 6.6 Slide-Based Q&A
- 6.7 Likely Exam Questions
- 6.8 Common Mistakes

Topic 7: The Three Named Reductions (Conceptual Understanding Only — No Proofs Required)

- 7.1 Independent Set $\rightarrow$ Vertex Cover
 - 7.1.1 The Two Problems
 - 7.1.2 What Direction Is This Reduction?
 - 7.1.3 The Core Idea (Intuitive)
 - 7.1.4 What Happens to the Instance and Solution
 - 7.1.5 Why Is This Reduction Useful?
 - 7.1.6 Common Traps for This Reduction
 - 7.1.7 Conceptual Exam Questions (No Proof Required)
- 7.2 3-SAT $\rightarrow$ Independent Set
 - 7.2.1 The Two Problems
 - 7.2.2 Direction
 - 7.2.3 The Core Idea (Intuitive, No Proof)
 - 7.2.4 What the Construction Represents
 - 7.2.5 The Two Directions of the Equivalence (Conceptual)
 - 7.2.6 Why This Reduction Matters / Is Useful
 - 7.2.7 Conceptual Exam Questions (No Proof Required)
 - 7.2.8 Common Traps for This Reduction
- 7.3 Vertex Cover $\rightarrow$ Set Cover
 - 7.3.1 The Two Problems
 - 7.3.2 Direction
 - 7.3.3 The Core Idea (Intuitive, No Proof)
 - 7.3.4 The Two Directions of the Equivalence (Conceptual)
 - 7.3.5 What Happens to the Instance and the Solution
 - 7.3.6 Why This Reduction Matters / Is Useful
 - 7.3.7 Conceptual Exam Questions (No Proof Required)
 - 7.3.8 Common Traps for This Reduction
- 7.4 Summary Table: All Three Reductions at a Glance
- 7.5 General Conceptual Questions That Apply to All Three Reductions

Topic 8: A Closing Note — HAM CYCLE and TSP

- 8.1 What the Slide Says
- 8.2 Simple Explanation
- 8.3 The Key Reminder: “Do Not Forget to Prove TSP Is in NP”
- 8.4 Likely Exam Questions
- 8.5 Common Mistakes

Master List: Definitions to Memorize

- Decision Problem
- P
- NP
- co-NP
- Complement of a Problem (X')
- Polynomial-time Reduction ($A \leq_p B$)
- NP-hard
- NP-complete
- Vertex Cover (decision problem)
- Independent Set (decision problem)
- Set Cover (decision problem)

Master List: Theorems & Key Results

Master List: Common Exam Traps (Consolidated)

How to Write Computational Complexity Answers (Templates)

- Template: “Define X”
- Template: “Explain X” / “Explain why X”
- Template: “Differentiate X and Y”
- Template: “Why is X important / useful?”
- Template: “Explain a reduction” (proof NOT required, e.g., the three named reductions)
- Template: “Prove that $X \subseteq Y$ ” (short set-containment proofs, e.g., $P \subseteq NP$, $P \subseteq \text{co-NP}$)
- Template: “What can you conclude if...?” (implication-chasing questions)

Final Last-Minute Revision Sheet

MUST MEMORIZE

MUST UNDERSTAND (don't need word-perfect recall)

REDUCTIONS (conceptual only — no proof needed)

COMMON TRAPS (top 5 to avoid)

TOP EXAM QUESTIONS TO REVIEW RIGHT BEFORE THE EXAM

How to Use This Guide

This guide is built **only** from your lecture slides (“CSE 207: Computational Complexity”). It does not add outside syllabus material. Every concept on the slides is explained here in plain language, then turned into exam questions with model answers.

Your teacher will **not** ask you to reproduce the proofs of these three reductions:

- Independent Set $\rightarrow$ Vertex Cover
- 3-SAT $\rightarrow$ Independent Set
- Vertex Cover $\rightarrow$ Set Cover

You still need to understand **what they mean conceptually** — this guide covers that in depth in the Reductions chapter, without asking you to memorize the constructions.

Legend used throughout:

- **VERY IMPORTANT** — must be ready to answer this cold.
- **IMPORTANT** — high chance of appearing in some form.
- **IF TIME** — good to know, lower priority with one day to prepare.

Notation used consistently, matching the slides:

- X' means the complement/negation of X (e.g., a variable’s negation, or a problem’s complement).
- $A \leq_p B$ means “A is polynomial-time reducible to B.”
- $\in$ means “is a member of” (e.g., $x \in P$ means x is a problem in class P).
- $\exists$ = “there exists”, $\forall$ = “for all”.

Table of Contents (Topic Map)

1. The Big Picture: Why Complexity Theory Exists
2. Decision Problems vs. Optimization Problems
3. The Classes P and NP
4. Complement of a Problem and co-NP
5. Polynomial-Time Reductions
6. NP-hard and NP-complete Problems
7. The Three Named Reductions (Conceptual Only)
8. A Closing Note: HAM CYCLE and TSP
9. Master List: Definitions to Memorize
10. Master List: Theorems & Key Results
11. Master List: Common Exam Traps
12. How to Write Computational Complexity Answers (Templates)
13. Final Last-Minute Revision Sheet

Topic 1: The Big Picture — Why Complexity Theory Exists

1.1 Simple Explanation

Many real problems ask you to pick the *best* option out of an enormous number of possibilities — this is called a **search problem** or **optimization problem**. Examples from the slides:

- **Shortest path**: find the shortest route between two nodes in a graph.
- **Minimum spanning tree**: find the cheapest set of edges connecting all nodes.

Surprisingly, both of these have fast (polynomial-time) algorithms. But the slides stress: **these are exceptions, not the rule**. For a huge number of other natural problems (e.g., satisfiability, vertex cover, set cover, TSP), nobody has ever found an efficient algorithm — despite decades of effort by brilliant people.

“**Efficient algorithm**” on the slides is defined as one that runs in **$O(n^k)$ time** — polynomial time — for some constant k . This is treated as the norm/standard for “efficient.”

1.2 Why This Matters — The Core Puzzle

The slides pose the deep question directly:

“How do you know there is no efficient solution (like the lower bound on comparison-based sorting)?”

This is illustrated with a cartoon (Garey & Johnson’s *Computers and Intractability*): an employee tells the boss either: - “I cannot solve it efficiently because I am dumb!” (personal failure), **or** - “I cannot find an efficient solution because there isn’t any!” (the problem itself is inherently hard).

The second explanation is much more acceptable — **but proving there is no efficient algorithm for a problem is extremely hard**. It’s very different from just failing to find one.

Intuitive analogy: Imagine you spent a year trying and failing to build a perpetual motion machine. You can’t personally *prove* it’s impossible just because you failed — that requires deep physics (laws of thermodynamics). Similarly, failing to find a fast algorithm doesn’t prove none exists — that requires deep theoretical tools.

1.3 The Strategy Complexity Theory Uses

Since directly proving “no efficient algorithm exists” is (usually) beyond our current mathematical tools, complexity theory instead asks a *relative* question:

Show that a problem is as hard as many other problems that brilliant scientists have already failed to solve efficiently, for a very long time.

This is the core idea behind: - Grouping problems into **equivalence classes of difficulty**. - **Comparing relative difficulty** of different problems (via reductions — Topic 5). - Calling a problem **NP-complete** to mean “as hard as the hardest problems in NP” (Topic 6).

So instead of proving “P is dumb, this problem has no fast algorithm,” we prove “this problem is at least as hard as thousands of other famous unsolved problems — so if you find a fast algorithm for this one, you’ve also solved all of them.”

1.4 Slide-Based Q&A

Q: What is a search/optimization problem? Give two examples from the slides. A: A problem where you must find the best (optimal) solution among an exponentially large set of candidate solutions. Examples: shortest path (minimize path length between two nodes) and minimum spanning tree (minimize total edge weight connecting all nodes).

Q: Are shortest path and MST typical of search problems in general? A: No — they are notable *exceptions*. Most search/optimization problems of this flavor do **not** have known efficient (polynomial-time) algorithms.

Q: What is meant by an “efficient algorithm” in this course? A: An algorithm with worst-case running time $O(n^k)$ for some constant k — i.e., polynomial time in the input size n . This is treated as the standard definition of “efficient” for this course.

Q: Why is it hard to prove that no efficient algorithm exists for a problem? A: Because failing to find a fast algorithm after trying is not the same as *proving* none exists. A rigorous impossibility proof (like the $\Omega(n \log n)$ lower bound for comparison-based sorting) requires a formal argument bounding *every possible algorithm*, not just the ones we’ve tried. For most hard problems, no such lower-bound proof is known.

1.5 Likely Exam Questions

● **VERY IMPORTANT | Easy Q**: What does it mean for an algorithm to be “efficient” in the context of this course? A: It runs in polynomial time, i.e., $O(n^k)$ for the input size n and some constant k .

● **VERY IMPORTANT | Medium Q**: Explain, using the “dumb employee” analogy, the difference between “I couldn’t solve it” and “it cannot be solved efficiently.” A: Not finding a fast algorithm despite effort only shows personal/collective failure so far; it does not prove

impossibility. Proving impossibility needs a rigorous argument that rules out *every* possible efficient algorithm — this is what complexity theory tries to approximate through relative-hardness arguments (reductions) since direct impossibility proofs are usually unavailable.

⊗ **IMPORTANT | Medium** Q: What is complexity theory’s main workaround for not being able to directly prove “no efficient algorithm exists”? A: Instead of proving absolute impossibility, we show a problem is *at least as hard as* many other long-studied problems (via polynomial-time reductions), placing it in an equivalence class of “equally hard” problems (e.g., NP-complete). If any one of them had an efficient algorithm, they all would.

⊗ **IF TIME | Easy** Q: Name two problems mentioned as having known polynomial-time solutions despite being search problems. A: Shortest path and Minimum Spanning Tree.

1.6 Common Mistakes

- Don’t say “shortest path is NP-hard” — it’s actually solvable in polynomial time (it’s the *exception*, not the rule).
- Don’t confuse “we haven’t found an efficient algorithm” with “it’s proven no efficient algorithm exists” — these are very different epistemic claims. Complexity theory almost never proves the second kind of statement for problems like SAT.

Topic 2: Decision Problems vs. Optimization Problems

2.1 Simple Explanation

A **decision problem** is a problem whose answer is always a single boolean value: **YES** or **NO**. This framework (decision problems) is what complexity theory (P, NP, NP-complete, etc.) is built on — it is much easier to formally analyze YES/NO questions than “find the best solution” questions.

Formal definition (from slides): A decision problem is a problem whose output is a single boolean value: YES or NO.

2.2 Converting Optimization → Decision

The slides show how to convert an optimization problem into an equivalent decision problem by adding a **threshold/bound**.

Optimization version	Decision version
Shortest path: Find the shortest path between s and t in graph G	Is there a path P between s and t in G such that $p \leq x$ (for a given bound x)?
SAT (as stated): Given a CNF Boolean formula, find a satisfying assignment	Is there any satisfying assignment? (YES/NO)

Why is the bound x required? Without a bound, “is there a path?” is trivially about connectivity, not about *optimality*. The bound turns “what is the minimum path length?” (a number-valued/optimization question) into “is the minimum path length at most x ?” (a YES/NO question). This is exactly what lets us fit the shortest path problem into the P-vs-NP framework, which is defined over decision problems.

2.3 Why It Matters — Optimization $\napprox$ Decision Are Equally Hard

The slides make an important equivalence point:

“Solving the optimization problem readily solves the decision problem.” (Find the shortest path, then just check if it’s within budget.)

“Turning an optimization problem into a search/decision problem does not necessarily change its difficulty — they reduce to one another.”

Direction 1 (Optimization $\Rightarrow$ Decision is easy): If you can *find* the shortest path, you trivially know whether it’s $\leq x$ — just compute the length and compare.

Direction 2 (Decision $\Rightarrow$ Optimization, via repeated queries): You can also recover the actual shortest path length by repeatedly asking the decision version with different bounds x (e.g., binary search over x , or incrementally trying $x = 1, 2, 3, \dots$ until the answer flips from NO to YES). This is the “how do you see it?” question the slides pose — the decision oracle can be called multiple times to *pin down* the optimal value, and with extra bookkeeping (e.g., trying to remove edges and re-querying) you can even reconstruct the actual path.

Key takeaway: Because optimization and decision versions reduce to each other (with only polynomial overhead), complexity theory can study *only* decision problems without losing generality about the difficulty of the original optimization problem.

2.4 Likely Exam Questions

● **VERY IMPORTANT | Easy Q:** Define a decision problem. A: A problem whose output is a single boolean value — YES or NO.

● **VERY IMPORTANT | Medium Q:** Convert the “shortest path” optimization problem into a decision problem, and explain why a bound is required. A: Decision version: “Given graph G, nodes s and t, and a number x, is there a path P from s to t with length $p \leq x$?” A bound x is required because without it the question collapses into a simple connectivity question (does *any* path exist?) and loses the notion of *optimality* that the original problem cares about. The bound lets us ask a YES/NO question that still captures “how good is the best solution.”

● **IMPORTANT | Medium Q:** Explain why solving the optimization version of a problem also solves its decision version, but not obviously the reverse. A: If you have an algorithm that outputs the optimal solution (e.g., the actual shortest path and its length), you can immediately answer any decision version query (is length $\leq x$?) by comparing the computed optimal value to x — this is trivial, one extra comparison. The reverse is less obvious: a decision oracle only answers YES/NO for a *given* x , so to recover the actual optimal value you need to invoke it multiple times (e.g., binary search on x), and to recover the actual solution (not just its value) may need further work (e.g., testing which edges can be removed while keeping the answer YES). The slides note they *do* reduce to each other, just via this extra step.

● **IF TIME | Hard Q:** Given only a “YES/NO: is there a vertex cover of size $\leq k$?” oracle, briefly describe how you might find an actual minimum vertex cover of a graph. A: First, binary search / increment k to find the smallest k for which the answer is YES — this gives the minimum vertex cover *size*. Then, to find the actual set, repeatedly test removing candidate vertices (or fixing them in/out of the cover) and re-querying the oracle with the same k to see whether a valid cover of that size still exists without that vertex — narrowing down to a concrete cover. (This is a reasoning-based extension of the slide’s point, not something explicitly worked out in the slides.)

2.5 Common Mistakes

- Don’t think decision problems are “less important” or a simplification that loses information — they are a *deliberate modeling choice* that doesn’t change the underlying difficulty (because they reduce to the optimization version and vice versa).

- Don't forget the bound x when converting an optimization minimization/maximization problem into decision form — leaving it out is one of the most common small mistakes.

Topic 3: The Classes P and NP

3.1 Formal Definitions

P (from slides): *The set of decision problems that can be solved in polynomial time.* Formally: given an input (instance) I , there is an algorithm that directly computes the correct YES/NO answer in polynomial time.

NP (from slides): *The set of decision problems with a polynomial-time **verification** for a “YES” answer.* Formally: given an input I and a proposed solution/certificate S , we can efficiently (in polynomial time) check whether S indeed corresponds to a “YES” answer.

3.2 Simple, Intuitive Explanation

- **P = “easy to solve.”** You have a recipe (algorithm) that, from scratch, produces the correct answer quickly.
- **NP = “easy to check” (given the answer).** You might have *no idea* how to find a solution quickly, but if someone *hands you* a candidate solution (a “certificate” or “witness”), you can quickly verify whether it’s correct.

Slide’s own phrase: “Hard to find, but easy to recognize!” — this is the heart of NP.

Concrete example from slides — Boolean Satisfiability (SAT): Formula: $(X \vee Y \vee Z) \wedge (X' \vee Y' \vee Z)$ - Given a candidate assignment S : $X=1, Y=0, Z=1$, we can plug in the values and check in polynomial time whether every clause evaluates to true. This check is fast $\implies$ **SAT $\in$ NP.** - However, there is **no known efficient algorithm** (as of today) to *find* such a satisfying assignment from scratch (or to verify a “NO” — see Topic 4). So SAT **may or may not** be in P — nobody knows.

Intuitive analogy: Think of a giant sudoku puzzle. Solving it from a blank grid might take a long time (brute-force search). But if someone hands you a *completed* grid, checking whether it’s a valid solution (no repeated digits in rows/columns/boxes) is fast. That’s the “hard to find, easy to check” idea behind NP.

3.3 $P \subseteq NP$ (Every Problem in P Is Also in NP)

Theorem (slides): P is a subset of NP. Formally: $x \in P \implies x \in NP$.

Why (slide’s proof idea): If there’s a polynomial-time algorithm A that *solves* problem x from scratch, then we can build a valid **certifier** B for x that simply **ignores the certificate entirely**, runs A on the input, and returns A ’s answer. Since A itself is already efficient, this “lazy” certifier B is also efficient. So every problem solvable in polynomial time trivially has a polynomial-time verification procedure too (you don’t even need to look at the proposed certificate — you can just solve it yourself).

Intuition: If you can already cook the meal yourself quickly, you don’t need someone else’s recipe (certificate) to confirm it’s a valid meal — you can just cook it again and compare.

3.4 Does $NP \subseteq P$? (Is $P = NP$?)

The slides pose two central open questions:

1. **Is NP a subset of P? (Is $P = NP$?)**
2. **Is there a problem in NP that does *not* belong to P? (Is $P \neq NP$?)**

Nobody knows the answer. This is one of the most famous open problems in computer science / mathematics (a Millennium Prize Problem). The slides show a Venn diagram of “what we think the world looks like”: **P is drawn as a strict subset inside NP**, reflecting the widespread *belief* (not proof) that $P \neq NP$.

3.5 Slide-Based Q&A

Q: Define P. A: The set of decision problems solvable by a polynomial-time algorithm.

Q: Define NP. A: The set of decision problems for which a proposed “YES” solution (certificate) can be verified in polynomial time.

Q: Why is SAT in NP? A: Given a candidate variable assignment, we can substitute the values into the CNF formula and check whether every clause is satisfied — this check takes polynomial time in the size of the formula.

Q: Why is SAT not (currently) known to be in P? A: No polynomial-time algorithm has ever been found that *constructs* a satisfying assignment (or determines none exists) from scratch, for arbitrary CNF formulas — despite extensive research. This does not mean SAT is proven to be outside P; it’s simply unknown/unsolved (P vs NP is open).

Q: Prove $P \subseteq NP$. A: Take any problem $x \in P$. By definition there is a polynomial-time algorithm A solving x directly. Define a certifier B for x that ignores whatever certificate it’s given, runs A on the instance, and outputs A ’s result. B runs in polynomial time (since A does) and correctly identifies YES instances (since A is correct). Hence x has a valid polynomial-time certifier, so $x \in NP$. Since x was arbitrary, $P \subseteq NP$.

3.6 Likely Exam Questions

● **VERY IMPORTANT | Easy Q:** State the definitions of P and NP. A: **P:** decision problems solvable in polynomial time. **NP:** decision problems whose YES-answers can be *verified* in polynomial time given a certificate.

● **VERY IMPORTANT | Medium Q:** Prove that $P \subseteq NP$. A: (See §3.5 proof above — must mention: take $x \in P$, use A (the poly-time solver)

to build a certifier B that ignores the certificate and just re-solves the problem, hence B verifies in poly time, so $x \in \text{NP}$.)

● **VERY IMPORTANT | Medium** Q: Explain why SAT is in NP but its membership in P is unknown. A: Verification is easy (plug in the assignment, check all clauses — polynomial time), which places SAT in NP by definition. But *finding* a satisfying assignment from scratch has no known efficient algorithm — this doesn't prove impossibility, it's simply an open/unsolved question, hence "SAT $\in$ P?" is unknown (in fact SAT is now known to be NP-complete — see Topic 6).

⊗ **IMPORTANT | Hard** Q: "If $\text{NP} \subseteq \text{P}$, what would that imply about P and NP?" A: We already know $\text{P} \subseteq \text{NP}$ always. If additionally $\text{NP} \subseteq \text{P}$, then $\text{P} = \text{NP}$ — every problem whose solution is easy to *verify* would also be easy to *solve* from scratch. This is one of the two central open questions in complexity theory.

● **IF TIME | Medium** Q: Give an intuitive real-world analogy for the difference between P and NP. A: P is like being able to *solve* a jigsaw puzzle yourself quickly. NP is like being handed a *completed* jigsaw puzzle and being able to quickly check that all the pieces fit correctly — even if you'd have struggled to assemble it yourself from scratch.

3.7 Common Mistakes / Traps

- **NP does NOT mean "Not Polynomial."** This is one of the most common misconceptions. NP stands for **Nondeterministic Polynomial (time)** — it is about *polynomial-time verification*, not about a problem being outside P. In fact $\text{P} \subseteq \text{NP}$, so many "easy" problems are also in NP.
- Do not say " $\text{P} = \text{NP}$ is proven false." It is **not proven** — it is only *widely believed* to be false ($\text{P} \neq \text{NP}$), based on the Venn diagram picture the slides show, but this remains an open problem.
- Do not confuse "solving" and "verifying." Solving = producing the answer/certificate from scratch. Verifying = checking a *given* certificate is valid. NP is defined via verification, not solving.
- Remember direction: $\text{P} \subseteq \text{NP}$ is **proven** (Topic 3.3). $\text{NP} \subseteq \text{P}$ (equivalently $\text{P} = \text{NP}$) is **open/unknown**.

3.8 Practice (True/False with reasoning)

1. "Every problem in NP is also in P." — **False (unknown, in fact widely believed false)**. This would mean $\text{P} = \text{NP}$, which is unproven and widely doubted.
2. "Every problem in P is also in NP." — **True**. Proven in §3.3.
3. "If a problem has no known polynomial-time algorithm, it is not in NP." — **False**. NP is about verification, not about solving. A problem can be very hard to solve but still easy to verify (e.g., SAT), placing it in NP regardless of whether an efficient solving algorithm is known.

Topic 4: Complement of a Problem and co-NP

4.1 Simple Explanation

For every decision problem X , there is a natural **complementary problem X'** : whenever the answer to X is YES, the answer to X' is NO, and vice versa.

Slide's example: - **SAT**: Is a given CNF Boolean formula satisfiable? - **UNSAT (= SAT')**: Is a given CNF Boolean formula **unsatisfiable**?

If SAT says YES for a formula, UNSAT says NO for that same formula, and vice versa.

4.2 Complement and P: $x \in P \Rightarrow x' \in P$

Theorem (slides): If a problem x is in P , then its complement x' is also in P . (**P is closed under complementation.**)

Why: If we have an efficient algorithm A solving x , we can build an algorithm A' for x' that simply **runs A and flips the answer** ($YES \leftrightarrow NO$). Since flipping a boolean output takes constant extra time, A' is still polynomial time. So $x' \in P$ too.

4.3 Does the Same Trick Work for NP? — No!

Key question the slides raise: Is it true that $x \in NP \Rightarrow x' \in NP$?

Answer: We don't know it's always true, and the reasoning that worked for P does NOT carry over.

Why the P-trick fails for NP: Having an efficient *certifier* for X (i.e., a fast way to verify a YES answer given a certificate) does **not** automatically give you an efficient certifier for X' (verifying a NO answer, i.e., a YES answer of X'). This is because of a fundamental asymmetry in how YES and NO are verified:

- **YES answers** (for many NP problems like SAT) are verified via an **"there exists"** quantifier — e.g., "there exists an assignment that satisfies the formula" — and you only need to exhibit **one** witness.
- **NO answers** require a **"for all"** quantifier — e.g., "for **all** possible assignments, the formula is not satisfied." Verifying a universal ($\forall$) statement in general seems to require checking *all* possibilities, which is not obviously doable with a short, quickly-checkable certificate.

This asymmetry between $\exists$ (existential, easy to certify with one witness) and $\forall$ (universal, hard to certify with a single short witness) is the crux of why NP is not obviously closed under complementation.

4.4 Definition of co-NP

co-NP (from slides): *The set of decision problems with a polynomial-time verification for a "NO" answer.* It is "just the opposite" of NP.

Formally: $x \in NP \Rightarrow x' \in \text{co-NP}$ (by definition — co-NP is literally defined as the set of complements of NP problems).

Example: UNSAT (is a formula unsatisfiable?) is in co-NP, since $\text{UNSAT} = \text{SAT}'$, and $\text{SAT} \in \text{NP}$.

4.5 Worked Example: PRIME

- **PRIME:** Is a given number x prime?
- Verifying a **NO** answer (x is *not* prime) is easy: just exhibit a nontrivial factor, and check that it divides x — this puts PRIME in **co-NP**.
- Verifying a **YES** answer (x is prime) did **not** seem easy at first glance (how do you certify "no factor exists" quickly?) — yet, remarkably:
- **PRIME is actually in P!** (Proven by the AKS algorithm — Agrawal, Kayal, and Saxena.) This is presented as a famous, somewhat surprising result.

Why this example matters for the exam: It shows that a problem being in co-NP doesn't prevent it from also being in P — and it illustrates that intuition about what "should" be hard can be wrong (PRIME looked hard to solve directly but was eventually shown to be in P).

4.6 $P \subseteq \text{co-NP}$

Theorem (slides): P is a subset of co-NP: $x \in P \Rightarrow x \in \text{co-NP}$.

Two ways the slides justify this:

1. **Direct intuition:** If the answer to a problem is NO and you can *solve* it in polynomial time, you can verify the NO answer without even looking at any certificate — just re-run your solving algorithm. (Same "ignore the certificate, just solve it" trick as $P \subseteq \text{NP}$.)
2. **Chained reasoning:** $x \in P \Rightarrow x' \in P$ (closure of P under complement, §4.2) $\Rightarrow x' \in \text{NP}$ (since $P \subseteq \text{NP}$, §3.3) $\Rightarrow x \in \text{co-NP}$ (by definition of co-NP as complements of NP problems).

4.7 Does $\text{NP} = \text{co-NP}$?

Open question (slides): Does $\text{NP} = \text{co-NP}$? **"We don't know yet!"** This is explicitly stated as unresolved, just like P vs NP.

Widespread belief: $\text{NP} \neq \text{co-NP}$ (just like the belief that $P \neq \text{NP}$). The slides state: **"Proving $\text{NP} \neq \text{co-NP}$ is even a bigger step than proving $P \neq \text{NP}$."**

4.8 The Relationship Between P, NP, co-NP (Key Implication Chain)

Theorem (slides): $NP \neq co-NP \Rightarrow P \neq NP$. (Equivalently, the contrapositive: $P = NP \Rightarrow NP = co-NP$.)

Proof idea (from slides): Suppose $P = NP$. Since P is always closed under complementation (§4.2: P is closed under complementation), we get: $NP = P = \text{complement of } P = co-NP$. So if $P = NP$, then automatically $NP = co-NP$ too.

Contrapositive reading (this is the useful exam direction): If we could somehow prove $NP \neq co-NP$, that would immediately prove $P \neq NP$ as a corollary. This is why the slides call proving $NP \neq co-NP$ “an even bigger step” — it’s a *strictly stronger* statement that would resolve P vs NP along the way.

Supporting proofs shown on slides (both directions of $NP \subseteq co-NP$ and $co-NP \subseteq NP$, ASSUMING $P = NP$):

- **$NP \subseteq co-NP$** (under the assumption $P = NP$): $x \in NP \Rightarrow x \in P$ (by assumption $P=NP$) $\Rightarrow x' \in P$ (P closed under complement) $\Rightarrow x' \in NP$ ($P \subseteq NP$) $\Rightarrow x \in co-NP$ (definition of $co-NP$).
- **$co-NP \subseteq NP$** (under the assumption $P = NP$): $x \in co-NP \Rightarrow x' \in NP$ (definition of $co-NP$) $\Rightarrow x' \in P$ (by assumption) $\Rightarrow x \in P$ (P closed under complement) $\Rightarrow x \in NP$ ($P \subseteq NP$).

Together these two inclusions give $NP = co-NP$ whenever $P = NP$.

4.9 Slide-Based Q&A

Q: Define co-NP. A: The set of decision problems for which a “NO” answer can be verified (certified) in polynomial time.

Q: Explain why “ $x \in NP \Rightarrow x' \in NP$ ” is not obviously true, even though “ $x \in P \Rightarrow x' \in P$ ” is true. A: For P , we can simply re-run the same polynomial-time solving algorithm and flip the answer — solving x and solving x' take the same effort. For NP , membership is about *verification*, not solving, and verifying YES vs. verifying NO are fundamentally different tasks: YES-verification typically uses an existential (“there exists a witness”) certificate, which is easy to check by exhibiting one witness; NO-verification typically requires a universal (“for all possible witnesses, none works”) argument, which doesn’t obviously reduce to checking one short certificate. So an efficient YES-certifier for X doesn’t hand you an efficient NO-certifier (i.e., a certifier for X').

Q: Is UNSAT in NP or co-NP? Why? A: UNSAT is in **co-NP**, because UNSAT is the complement of SAT, and $SAT \in NP$; by definition, complements of NP problems are exactly the problems in $co-NP$.

Q: What does the PRIME example teach us? A: PRIME (is x prime?) is in $co-NP$ because “NO” (composite) is easy to verify by exhibiting a factor. It was *not* obviously in NP or P for a long time. Eventually, the AKS algorithm showed $PRIME \in P$ — proving that a problem can be efficiently solvable even when it initially seems to only have an easy *complement*-verification. It’s a caution against assuming a problem is hard just because an efficient algorithm hasn’t obviously appeared yet.

4.10 Likely Exam Questions

● **VERY IMPORTANT | Medium** Q: Prove that P is closed under complementation. A: Let $x \in P$, so there is a polynomial-time algorithm A that solves x . Construct A' for x' that runs A on the same input and outputs the opposite boolean (flip YES $\leftrightarrow$ NO). A' runs in polynomial time (A ’s time plus a constant), and correctly solves x' by definition of complement. Hence $x' \in P$.

● **VERY IMPORTANT | Medium** Q: Prove $P \subseteq co-NP$. A: Let $x \in P$. Then $x' \in P$ (P closed under complementation). Since $P \subseteq NP$, $x' \in NP$. By definition, $co-NP$ is the set of complements of NP problems, so since $x' \in NP$, its complement $x = (x')' \in co-NP$.

● **VERY IMPORTANT | Hard** Q: Show that $P = NP$ implies $NP = co-NP$. A: Assume $P = NP$. Since P is always closed under complementation, taking complements of P gives P itself back: complement of $P = P$. So we can write the chain $NP = P = \text{complement of } P = co-NP$, i.e., $NP = co-NP$.

● **IMPORTANT | Medium** Q: What can we conclude if someone manages to prove $NP \neq co-NP$? A: By the contrapositive of “ $P = NP \Rightarrow NP = co-NP$,” proving $NP \neq co-NP$ would immediately imply $P \neq NP$. This is why the slides describe proving $NP \neq co-NP$ as “an even bigger step” than proving $P \neq NP$ — it’s a strictly stronger, harder statement.

● **IMPORTANT | Medium** Q: Why is verifying a “NO” answer for SAT (i.e., proving unsatisfiability) considered hard in general, using quantifier language? A: A “YES” answer to SAT is verified by exhibiting one satisfying assignment (an $\exists$ -statement: “there exists an assignment that works”), which is a short, quickly checkable certificate. A “NO” answer means “for all possible assignments, the formula fails” (a $\forall$ -statement over an exponential number of assignments) — there’s no known short certificate that lets you skip checking all of them, so no known efficient certifier for the NO case exists in general.

● **IF TIME | Easy** Q: Is PRIME in P ? Who showed this? A: Yes — $PRIME \in P$, proven by Agrawal, Kayal, and Saxena (the “AKS algorithm”).

4.11 Common Mistakes

- Don’t assume NP is closed under complement “by analogy” with P — this is exactly the trap the slides warn about. **P is closed under complement (proven); NP being closed under complement is unknown (open question, in fact believed false).**
- Don’t mix up the direction of the implication: it’s **$P=NP \Rightarrow NP=co-NP$** , not the other way around as a proof technique (though logically $NP \neq co-NP \Rightarrow P \neq NP$ is the useful *contrapositive*, which is a valid restatement, not a separate independent theorem).
- Remember: $co-NP$ is **defined structurally** as “complements of NP problems,” and *separately* characterized as “problems with poly-time NO-verification.” Both descriptions are equivalent and both appear on slides — know both phrasings.
- $UNSAT \in co-NP$, **not** $UNSAT \in NP$ (unless $P = NP$, which is unproven) — don’t casually say “UNSAT is in NP .”

Topic 5: Polynomial-Time Reductions

5.1 Formal Definition and Intuitive Meaning

Notation: $A \leq_p B$ means “Problem A is polynomial-time reducible to problem B.”

Intuitive meaning: You can solve A by (1) transforming any instance of A into an instance of B (using a polynomial-time transformation), (2) feeding that transformed instance into a “black box” that solves B, and (3) translating B’s answer/solution back into an answer/solution for the original instance of A — all of this using only polynomial extra work.

Diagram from slides (Algorithm for A, built out of an algorithm for B):

```
Instance I --[f]--> Instance f(I) --[Algorithm for B]--> Solution S of f(I) --[h]--> Solution h(S) of I
(or: "No solution to f(I)" --> "No solution to I")
```

- f = the polynomial-time transformation function that converts an A-instance into a B-instance.
- The “Algorithm for B” is treated as a **black box** — we don’t care how it works internally, only that it correctly solves B.
- h = a polynomial-time function that translates B’s solution back into a valid solution for the original A-instance.

Analogy: Suppose you don’t know how to solve Sudoku, but you know a general “constraint satisfaction solver” black box. If you can quickly *translate* any Sudoku puzzle into an equivalent constraint-satisfaction instance, and quickly translate the solver’s answer back into a filled Sudoku grid, then you’ve “reduced” Sudoku to constraint satisfaction — you can solve Sudoku *using* the constraint solver, without needing to understand Sudoku-solving from scratch.

5.2 Why Reductions Matter: Difficulty and Efficiency Flow in Opposite Directions

Core statements straight from the slides (memorize these):

“A is at least as hard as B” — Wait, careful: the slide actually marks “**B is at least as hard as A**” with a checkmark as the correct one to remember; “A is at least as hard as B” is shown crossed out as the wrong direction to focus on. (See §5.3 “Common Mistakes” — this exact reversal is explicitly the trap the slides warn about in the very next section, “Reduction — Warning.”)

To avoid confusion, use these two clean, unambiguous statements instead, both explicitly given on the slides:

“If you can solve B in polynomial time, then you can solve A in polynomial time.”

“If you cannot solve A in polynomial time, then you cannot solve B in polynomial time.” (this is just the contrapositive of the line above)

“Difficulty flows from A to B; Efficiency flows from B to A.”

How to read “difficulty flows from A to B”: If $A \leq_p B$, then B is “at least as hard” as A in the sense that B being easy would make A easy too. So the “hardness” of A gets passed onto B (B inherits at least A’s difficulty) — while an efficient algorithm for B flows back and gives you efficiency for A.

In plain words: $A \leq_p B$ tells you “**B is at least as hard as A**.” If you ever find a fast algorithm for B, you immediately get a fast algorithm for A for free (just chain f , B’s algorithm, and h). Conversely, if you know for certain A is hard (no fast algorithm exists), then B must also be hard — because if B were easy, that would make A easy too (contradiction).

5.3 The Warning Slide — Direction Matters!

The slides include an entire slide titled “**Reduction — Warning**”:

“You should reduce the **known** NP-complete problem **to** the problem you are trying to prove NP-complete. You will mistakenly do the opposite sometimes!”

Why this matters so much: To prove a *new* problem X is NP-hard, you must show $(\text{known NP-complete problem}) \leq_p X$ — i.e., reduce FROM the already-known-hard problem TO the new problem. Students very commonly get this backwards and instead reduce X to the known problem, which does **not** prove what they intended.

Memory trick: You’re trying to show X is *at least as hard as* the known hard problem. Reductions push difficulty *forward* ($A \leq_p B$ means $B \geq A$ in hardness). So to make X “inherit” the known problem’s hardness, the known problem must play the role of A, and X must play the role of B: $(\text{known-hard}) \leq_p X$.

5.4 Slide-Based Q&A

Q: What does $A \leq_p B$ mean? A: Problem A is polynomial-time reducible to problem B: there is a polynomial-time transformation that turns any instance of A into an equivalent instance of B, such that solving the B-instance (via a hypothetical black-box algorithm) and translating back gives a correct solution to the original A-instance, using only polynomial-time extra work for the transformation and back-translation.

Q: If $A \leq_p B$ and B can be solved in polynomial time, what can you conclude about A? A: A can also be solved in polynomial time (by transforming to B, solving B, and translating back — all polynomial steps compose to a polynomial-time algorithm for A).

Q: If $A \leq_p B$ and A is known to have no efficient algorithm, what can you conclude about B? A: B also has no efficient algorithm (polynomial-time). If B did have one, we could use the reduction to build an efficient algorithm for A too, a contradiction.

Q: Why must you reduce a *known* NP-complete problem to a *new* problem (rather than the reverse) to prove the new problem is NP-hard? A: Because reductions transfer hardness in the direction of the arrow: $A \leq_p B$ shows B is at least as hard as A. To show a new problem X is at least as hard as an already-known-hard problem, the known-hard problem must be A (the source) and X must be B (the target): $(\text{known NP-complete}) \leq_p X$. Reducing in the opposite direction would only show that X is *no harder* than the known problem (or say nothing useful about X's hardness).

5.5 Likely Exam Questions

● **VERY IMPORTANT | Easy** Q: Define polynomial-time reduction ($A \leq_p B$). A: A is polynomial-time reducible to B if there exist polynomial-time computable functions f (mapping A-instances to B-instances) and h (mapping B-solutions back to A-solutions) such that solving $f(I)$ with any algorithm for B and applying h to the result gives a correct solution to the original A-instance I .

● **VERY IMPORTANT | Medium** Q: “Difficulty flows from A to B; efficiency flows from B to A.” Explain this statement. A: If $A \leq_p B$: any efficient algorithm we discover for B can be converted (via the reduction) into an efficient algorithm for A — so *efficiency* discovered at B “flows back” to benefit A. Conversely, if A is known to be hard (inherently difficult), that hardness “flows forward” onto B, because B must be at least as hard as A (otherwise A would become easy too, via the reduction) — so *difficulty* flows from A to B.

● **VERY IMPORTANT | Hard** Q: Explain the mistake described in the “Reduction — Warning” slide and why it invalidates an NP-hardness proof. A: To prove a new problem X is NP-hard, one must reduce a *known* NP-complete problem TO X ($\text{known} \leq_p X$), showing X is at least as hard as an already-hard problem. A common mistake is instead reducing X to the known problem ($X \leq_p \text{known}$) — but this only shows X is *no harder* than something already known to be in NP (or doesn't show X is hard at all); it does not establish that X inherits the known problem's hardness. Getting the direction backwards produces a reduction that proves the wrong (often useless or trivially true) statement.

⊗ **IMPORTANT | Medium** Q: What role does the “black box” play in the definition of a reduction? A: The black box represents a hypothetical algorithm that solves problem B — we don't need to know or care how it works internally. The reduction only needs to show that *if* such a black box existed (efficient or not), it could be used (together with the polynomial-time transform/translate steps f and h) to solve A. This isolates the “transformation logic” of the reduction from the actual difficulty of solving B.

● **IF TIME | Easy** Q: In the diagram Instance $I \rightarrow f \rightarrow f(I) \rightarrow$ Algorithm for B $\rightarrow$ Solution S of $f(I) \rightarrow h \rightarrow$ Solution $h(S)$ of I , what do f and h represent? A: f transforms an A-instance into an equivalent B-instance (in polynomial time); h translates a solution of the B-instance back into a valid solution of the original A-instance (also in polynomial time).

5.6 Common Mistakes

- **Direction confusion** is the single biggest trap in this entire topic (see the explicit “Warning” slide). Always double check: to prove X is NP-hard, you reduce **FROM** a known-hard problem **TO** X, i.e. $\text{known} \leq_p X$.
- Do not think $A \leq_p B$ means A and B have “equal difficulty” — it only shows B is *at least* as hard as A (a one-directional inequality). Showing true equivalence in difficulty requires reductions in *both* directions.
- Don't forget that the transformation function f and the back-translation function h must **both** run in polynomial time — if either step were exponential, the overall reduction wouldn't preserve polynomial-time equivalence.

Topic 6: NP-hard and NP-complete Problems

6.1 NP-hard — Formal Definition

NP-hard (from slides): Problem X is NP-hard if **for all $Y \in \text{NP}$, $Y \leq_p X$** .

In words: X is at least as hard as *every single problem in NP* — every NP problem can be polynomial-time reduced to X.

Equivalent phrasing (slides): “A problem X is NP-hard if a polynomial-time algorithm for X would imply a polynomial-time algorithm for *every problem in NP*.”

Key nuance: NP-hard problems don’t have to be in NP themselves! NP-hard is only about being “at least as hard as everything in NP” — it says nothing about whether X’s own YES-answers can be efficiently verified. (This is an important, commonly tested distinction — see §6.4.)

6.2 NP-complete — Formal Definition

NP-complete (from slides): Problem X is NP-complete if: (i) **$X \in \text{NP}$** , and (ii) **for all $Y \in \text{NP}$, $Y \leq_p X$** (i.e., X is NP-hard).

In words: NP-complete = NP-hard **AND** also itself a member of NP.

Slide’s summary statements: > “NP-complete problems are the hardest problems in NP.” > > “NP-complete is a set of problems with the ‘same level of’ difficulty.”

Intuition: NP-complete problems form a tightly-linked club: any one of them reduces to any other (since each is NP-hard, meaning every NP problem — including every other NP-complete problem — reduces to it; and since they’re all in NP, each can serve as the “Y” being reduced too). If you ever find a polynomial-time algorithm for even **one** NP-complete problem, you would immediately get one for **every** problem in NP (since $P = \text{NP}$ would follow) — this is exactly why NP-completeness is such a big deal.

6.3 The Venn Diagram Picture (“What we think the world looks like”)

The slides show a nested picture:



More precisely, two separate diagrams are given: 1. **$P \subseteq \text{NP}$** : a small circle (P) inside a larger circle (NP) — reflecting the belief that P is a strict subset of NP. 2. **NP-complete sits at the intersection of NP and NP-hard**: NP-complete problems are exactly those that are *both* in NP and NP-hard. NP-hard, as a class, extends beyond NP (it includes problems that may be even harder than anything in NP, such as problems that aren’t even decidable, or optimization versions of NP-complete problems that aren’t decision problems at all).

Believed relationship (assuming $P \neq \text{NP}$, the standard picture): $P \subset \text{NP}$, and NP-complete problems sit “at the frontier” — they are the hardest problems inside NP, but NP-hard problems can lie entirely outside NP altogether (harder still).

6.4 Why “NP-hard” Is Not the Same as “NP-complete”

This distinction is one of the most commonly tested and commonly confused points in complexity theory courses in general, and is directly implied by the slide definitions:

	Must be in NP?	Must every NP problem reduce to it?
NP-hard	No (not required)	Yes
NP-complete	Yes (required)	Yes

Example type of problem that is NP-hard but *not* NP-complete: the **optimization version** of an NP-complete decision problem (e.g., “find the actual minimum vertex cover,” rather than “is there a vertex cover of size $\leq k$?”). The optimization version is at least as hard as the decision version (Topic 2 showed $\text{decision} \leq_p \text{optimization}$ essentially), so it’s NP-hard — but since its output isn’t a simple YES/NO, it doesn’t fit the definition of a decision problem at all, so it cannot be a member of NP (NP is defined only over decision problems), and hence can’t be NP-complete.

6.5 Proving NP-completeness — General Method

Slides’ statement: > “You have to prove that for all problems $Y \in \text{NP}$, $Y \leq_p X$.” — this is the literal definition, but proving it directly for *every* $Y \in \text{NP}$ is impractical. > > **Alternative: Prove that a known NP-complete problem $Y \leq_p X$.**”

Why the alternative works: If some already-known NP-complete problem Y satisfies $Y \leq_p X$, then since (by definition of Y being NP-complete) every problem in NP already reduces to Y, and reductions compose (if $Z \leq_p Y$ and $Y \leq_p X$, then $Z \leq_p X$), every problem in NP transitively reduces to X as well. So it suffices to reduce **one** known NP-complete problem to X, instead of proving the property from scratch for the infinite set of all NP problems.

Full recipe to prove X is NP-complete: 1. Show $X \in \text{NP}$ (give a polynomial-time verifier / certificate scheme for X). 2. Pick a **known NP-complete problem Y**. 3. Show $Y \leq_p X$ (construct a polynomial-time reduction from Y to X — remember the direction! See the Warning in §5.3).

6.6 Slide-Based Q&A

Q: Define NP-hard. A: A problem X is NP-hard if every problem Y in NP is polynomial-time reducible to X ($Y \leq_p X$ for all $Y \in \text{NP}$).

Q: Define NP-complete. A: A problem X is NP-complete if (i) $X \in \text{NP}$, and (ii) X is NP-hard (every $Y \in \text{NP}$ reduces to X).

Q: Why are NP-complete problems called “the hardest problems in NP”? A: Because by the NP-hard condition, every other problem in NP can be reduced to them (they are at least as hard as anything in NP) — while also being members of NP themselves, so they represent the hardest problems that are still “inside” the NP club.

Q: Why does it suffice to reduce one known NP-complete problem to X, instead of reducing every $Y \in \text{NP}$ to X? A: Reductions compose: if Y is NP-complete, every $Z \in \text{NP}$ already satisfies $Z \leq_p Y$. If we additionally show $Y \leq_p X$, then by transitivity $Z \leq_p X$ for every $Z \in \text{NP}$ — exactly the NP-hardness condition for X. So a single reduction from a known NP-complete problem stands in for infinitely many direct reductions.

Q: Can a problem be NP-hard without being NP-complete? Give the reasoning. A: Yes. NP-hard only requires “every NP problem reduces to it” — it does not require the problem itself to be in NP. A problem could fail to be in NP (e.g., it’s not even a decision problem, like an optimization problem, or its YES-answers have no known efficient certificate) while still being at least as hard as everything in NP. Such a problem is NP-hard but not NP-complete.

6.7 Likely Exam Questions

● **VERY IMPORTANT | Easy** Q: State the two conditions required for a problem to be NP-complete. A: (1) The problem must be in NP. (2) The problem must be NP-hard, i.e., every problem in NP must be polynomial-time reducible to it.

● **VERY IMPORTANT | Medium** Q: Differentiate NP-hard and NP-complete. A: NP-hard problems are “at least as hard as” every problem in NP (every NP problem reduces to them), but they need not themselves be in NP. NP-complete problems satisfy the *same* hardness condition **and** are additionally required to be in NP. So: $\text{NP-complete} = \text{NP-hard} \cap \text{NP}$. Every NP-complete problem is NP-hard, but not every NP-hard problem is NP-complete.

● **VERY IMPORTANT | Medium** Q: Describe the standard method used to prove a new problem X is NP-complete, and justify why it’s valid. A: Show $X \in \text{NP}$ (exhibit a polynomial-time verifier), then pick any already-known NP-complete problem Y and construct a polynomial-time reduction $Y \leq_p X$. This is valid because reductions compose: since Y is NP-complete, every $Z \in \text{NP}$ reduces to Y ($Z \leq_p Y$); combined with $Y \leq_p X$, transitivity gives $Z \leq_p X$ for every $Z \in \text{NP}$, satisfying the NP-hardness requirement for X. Together with $X \in \text{NP}$, this proves X is NP-complete.

● **IMPORTANT | Medium** Q: If a polynomial-time algorithm is ever found for a single NP-complete problem, what follows? A: Since every problem in NP reduces (in polynomial time) to that NP-complete problem, a polynomial-time algorithm for it would let us solve every NP problem in polynomial time as well — meaning $\text{NP} \subseteq \text{P}$, and since $\text{P} \subseteq \text{NP}$ always, this would prove $\text{P} = \text{NP}$.

● **IMPORTANT | Hard** Q: Give an example of a problem that could plausibly be NP-hard but not NP-complete, and explain why. A: The *optimization* version of an NP-complete problem — e.g., “output the size of the minimum vertex cover” (rather than the decision version “is there a vertex cover of size $\leq k$?”). It is at least as hard as the (already NP-hard) decision version, so it’s NP-hard, but its output is a number, not a YES/NO — it is not a decision problem at all, so it cannot belong to NP (which is only defined for decision problems), and hence cannot be NP-complete.

● **IF TIME | Easy** Q: Is every NP-complete problem also NP-hard? A: Yes, by definition — NP-hardness is one of the two required conditions for NP-completeness.

6.8 Common Mistakes

- **Saying “NP-hard” when you mean “NP-complete”, or vice versa** — remember NP-complete is strictly narrower (it requires membership in NP as well).
- **Believing all NP-hard problems are decision problems** — false; NP-hardness is defined via reductions from NP problems, and the target problem X does not need to be a decision problem itself (it just needs every NP problem to reduce to it).
- **Thinking NP-completeness must be proven by reducing to every NP problem individually** — the standard/practical method is to reduce a *single already-known* NP-complete problem to your new problem (transitivity does the rest).
- **Forgetting the “ $X \in \text{NP}$ ” part of the NP-completeness proof** — many students jump straight to the reduction and forget that showing membership in NP is a required, separate step.

Topic 7: The Three Named Reductions (Conceptual Understanding Only — No Proofs Required)

Reminder: Your teacher explicitly said you will **not** be asked to reproduce/prove these three reductions:

1. Independent Set $\rightarrow$ Vertex Cover
2. 3-SAT $\rightarrow$ Independent Set
3. Vertex Cover $\rightarrow$ Set Cover

But he **may still ask conceptual questions** about what each reduction means, why it works, what it establishes, and how to interpret a given construction. This chapter is built entirely around that kind of question. **Do not spend time memorizing the detailed proofs** — focus on the “what/why/so-what” for each.

7.1 Independent Set $\rightarrow$ Vertex Cover

7.1.1 The Two Problems

- **Vertex Cover:** Given a graph G and a number k , does G contain a vertex cover of size at most k ? (A vertex cover is a set S of nodes such that every edge has at least one endpoint in S — i.e., S “covers” every edge.)
- **Independent Set:** Given a graph G and a number k , does G contain a set of at least k independent vertices? (An independent set is a set I of nodes that are pairwise non-adjacent — no two nodes in I share an edge.)

7.1.2 What Direction Is This Reduction?

The slide title is “**Independent Set to Vertex Cover**” — meaning the connection is drawn as **Independent Set $\leftrightarrow$ Vertex Cover**, based on a shared underlying theorem rather than a one-way hardness reduction in the usual NP-completeness sense. The key content is actually an **iff (if and only if) relationship**, not a one-directional reduction of one to the other for hardness purposes:

Theorem: If $G = (V, E)$ is a graph, then S is an independent set **if and only if** $V - S$ (all vertices NOT in S) is a vertex cover.

7.1.3 The Core Idea (Intuitive)

An edge (u, v) is “covered” by a vertex cover if at least one of u, v is included. An edge (u, v) fails to be “independent-set-safe” if *both* u and v are in the independent set (since that would make them adjacent, violating independence). These are exactly **complementary conditions** over the same edge — which is why the *complement* of an independent set (all the vertices you *didn't* pick) turns out to be exactly a vertex cover, and vice versa.

Intuitive analogy: Imagine assigning every vertex a “chosen” or “not chosen” label. For each edge, “vertex cover” wants **at least one endpoint chosen**. “Independent set” wants **not both endpoints chosen**. If we let “chosen” = in the independent set, then “not both chosen” for every edge is exactly the same as saying “at least one endpoint is NOT chosen” — which describes a vertex cover, but made of the *unchosen* vertices.

7.1.4 What Happens to the Instance and Solution

- **Input transformation:** None needed — literally the **same graph G** is used for both problems! This is unusually simple compared to typical reductions.
- **Size relationship:** If $|V| = n$, and S is an independent set of size k , then $V - S$ is a vertex cover of size **$n - k$** . So “independent set of size $\geq k$ ” corresponds to “vertex cover of size $\leq n - k$.”
- **Solution translation:** Given an independent set solution S , the vertex cover solution is simply the complement $V - S$ (and vice versa) — no further work needed.

7.1.5 Why Is This Reduction Useful?

It establishes that **Independent Set and Vertex Cover are essentially two views of the same underlying structure** in a graph. If you can efficiently solve one, you can efficiently solve the other (just complement the solution set). It shows these two seemingly different graph problems (“find isolated nodes” vs. “find a small covering set”) are tightly linked — solving one for size k is equivalent to solving the other for size $n - k$.

7.1.6 Common Traps for This Reduction

- Don’t say the reduction “changes the graph” — it uses the **exact same graph**; only the *notion of k* and the *set itself* get complemented.
- Don’t confuse the direction of the size relationship: independent set size $k \leftrightarrow$ vertex cover size **$n - k$** (not size k).
- This is a rare case of an “iff” structural theorem being used as the basis of a reduction in *both* directions at once, rather than a one-way, complex gadget-based construction like the other two reductions below.

7.1.7 Conceptual Exam Questions (No Proof Required)

● **VERY IMPORTANT | Easy Q:** State the relationship between independent sets and vertex covers. A: For a graph $G = (V, E)$, a set S is an independent set if and only if $V - S$ is a vertex cover.

● **VERY IMPORTANT | Medium** Q: If a graph on 10 vertices has a maximum independent set of size 4, what is the size of its minimum vertex cover? A: $10 - 4 = 6$.

● **IMPORTANT | Medium** Q: Why does solving Independent Set efficiently also let you solve Vertex Cover efficiently (and vice versa)? A: Because of the iff theorem: a set is an independent set exactly when its complement is a vertex cover. So any algorithm that finds a maximum independent set S immediately gives you a minimum vertex cover $(V - S)$, with no extra transformation needed beyond computing the set complement, and vice versa — this translation is trivially fast (polynomial/linear time).

● **IF TIME | Medium** Q: Is the transformation used in this reduction complicated? Why or why not? A: No — it's unusually simple. Both problems are defined on the *same* graph; the only “transformation” is taking the set complement $(V - S)$, which is immediate once S is known. This is different from the more complex gadget-based constructions used in the other two reductions in this chapter.

7.2 3-SAT → Independent Set

7.2.1 The Two Problems

- **3-SAT:** An instance of 3-SAT has variables $X = \{x_1, x_2, \dots, x_n\}$ and k clauses $C_1, C_2, \dots, C_k$, where each clause is a disjunction (OR) of exactly 3 literals (a variable or its negation), and the whole formula is the AND (conjunction) of all clauses. Example from slides: $(x \vee y \vee z) \wedge (x' \vee y' \vee p) \wedge (y \vee z' \vee p')$. Question: is there an assignment of variables making the whole formula true?
- **Independent Set:** (defined in §7.1.1)

7.2.2 Direction

3-SAT $\leq_p$ Independent Set — this reduces the known NP-complete problem (3-SAT) to Independent Set, in order to help prove Independent Set is NP-hard (matches the correct direction described in the Warning in Topic 5: reduce FROM a known NP-complete problem TO the new one).

7.2.3 The Core Idea (Intuitive, No Proof)

The slides explicitly flag this as a “**non-trivial reduction, as the problems are quite different in nature**” (unlike §7.1, which reused the same graph).

Construction idea (conceptual level only): - For each clause (which has 3 literals), build a **triangle** (clique of 3 nodes) in the graph — one node per literal in that clause. - Add an **edge between x and x'** whenever both a variable and its negation appear (possibly in different clauses) — connecting “contradictory” literal nodes.

Why a triangle per clause? Within a triangle, at most **one** node can be picked into an independent set (since all three nodes are mutually adjacent — picking two would violate independence). This structurally forces “at most one literal chosen per clause” — mirroring the fact that a clause is *satisfied* as soon as **at least one** of its literals is true (we don't need more than one “witness” literal per clause).

Why an edge between x and x' ? This prevents the independent set from simultaneously choosing a literal and its negation from different clauses — because that would correspond to assigning a variable **both** true and false at once, which is logically inconsistent. The edge enforces consistency of the variable assignment across the whole formula.

7.2.4 What the Construction Represents

- **Each node** = one occurrence of a literal in a specific clause.
- **Each triangle (clique)** = one clause; choosing a node from a triangle for the independent set = “picking” that literal as the one that satisfies that clause.
- **Cross edges (x to x')** = consistency constraints, preventing contradictory choices across different clauses.

7.2.5 The Two Directions of the Equivalence (Conceptual)

- “**Satisfiable $\implies \exists$ an independent set**”: If the 3-SAT formula is satisfiable, every clause/triangle has at least one literal that evaluates to true (since the clause is an OR of 3 literals, and the clause must be true). Pick one true-evaluating node from each triangle — since no two picked nodes are ever connected by an edge (they don't share a triangle, and they can't be contradictory x/x' pairs because a consistent assignment doesn't set both x and x' true), this collection forms a valid independent set of size **k** (one node per clause, and there are k clauses).
- “ **$\exists$ an independent set of size $k \implies$ Satisfiable**”: Given an independent set of size exactly k , since a triangle can contribute at most 1 node, hitting size k forces **exactly one node from every single triangle/clause** to be chosen. We then read off an assignment: if a literal x 's node is in the set, set $x_i = 1$ (true); if x_i' 's node is in the set, set $x_i = 0$; if neither appears, the variable's value doesn't matter (assign freely). Because of the cross-edges preventing x and x' from both being chosen, this assignment is well-defined and consistent, and it satisfies every clause (since each clause's triangle contributed a “true” literal).

7.2.6 Why This Reduction Matters / Is Useful

It's one of the classic building blocks that shows **Independent Set is NP-hard** (since 3-SAT, a foundational NP-complete problem via the Cook-Levin theorem — background knowledge, though not stated explicitly on your slides — reduces to it). Combined with the fact that Independent Set $\in$ NP (a proposed set is trivially checkable in polynomial time: just verify no two chosen vertices share an edge, and count that there are $\geq k$ of them), this reduction is one ingredient in proving **Independent Set is NP-complete**.

7.2.7 Conceptual Exam Questions (No Proof Required)

● **VERY IMPORTANT | Easy** Q: In the 3-SAT to Independent Set construction, what does each triangle in the graph represent? A: Each triangle represents one clause of the 3-SAT formula (one node per literal in that clause).

● **VERY IMPORTANT | Medium** Q: Why can at most one vertex from each triangle be part of a valid independent set? A: Because all three vertices within a triangle are pairwise adjacent (fully connected), so picking any two of them would violate the independent-set condition (no two chosen vertices may be adjacent).

● **VERY IMPORTANT | Medium** Q: What is the purpose of the edges added between a literal x and its negation x' ? A: To enforce logical consistency — they prevent the independent set from simultaneously including both “ x is the satisfying literal” and “ x' is the satisfying literal” from different clauses, which would correspond to assigning the same variable both true and false at once.

● **IMPORTANT | Medium** Q: If a 3-SAT instance has k clauses and is satisfiable, what size of independent set does the construction guarantee, and why? A: Size k (one node per clause/triangle) — because a satisfying assignment makes every clause true via at least one of its literals, and picking that “witness” literal’s node from each of the k triangles gives k mutually non-adjacent nodes (no shared triangle, and no contradictory x/x' pair since the assignment is consistent).

● **IMPORTANT | Hard** Q: Explain, at a conceptual level (no proof), why “3-SAT $\leq_p$ Independent Set” is described as a “non-trivial” reduction, unlike the Independent Set $\leftrightarrow$ Vertex Cover relationship. A: Independent Set and Vertex Cover are almost the “same problem” phrased two ways on the identical graph — the transformation is trivial (just complement the vertex set). 3-SAT and Independent Set, by contrast, are structurally very different (one is about logical formulas and variable assignments; the other is about graph adjacency) — the reduction has to *build an entirely new graph from scratch* (via gadgets: triangles per clause, cross-edges for consistency) that has no obvious direct correspondence to the original formula’s structure. This kind of “gadget construction” reduction is considered non-trivial because it requires cleverly encoding logical structure (AND of ORs of literals) into purely combinatorial graph structure (adjacency/independence).

● **IF TIME | Medium** Q: Given an independent set found in the constructed graph, how would you recover a satisfying assignment for the original 3-SAT formula? A: For each variable x_i : if the node for literal x_i is in the independent set, set $x_i = 1$ (true); if the node for x_i' is in the set, set $x_i = 0$ (false); if neither of x_i ’s literal-nodes appears in the independent set, x_i ’s value can be assigned arbitrarily (it doesn’t affect satisfiability).

7.2.8 Common Traps for This Reduction

- Don’t think the reduction reuses/modifies the 3-SAT formula’s structure directly as a graph in a “natural” way — it’s a deliberately constructed gadget graph (triangles + cross edges), unlike §7.1’s trivial complement trick.
- Don’t forget the reduction direction: **3-SAT $\leq_p$ Independent Set** (3-SAT is the already-known-hard problem; Independent Set is the target being proven hard) — not the reverse.
- Remember the independent set size sought equals **k** , the number of clauses — not the number of variables.

7.3 Vertex Cover $\rightarrow$ Set Cover

7.3.1 The Two Problems

- **Vertex Cover:** (defined in §7.1.1)
- **Set Cover:** Given a set U of n elements and a collection of subsets $S_1, S_2, \dots, S_m$ of U , is there a collection of **at most k** of these subsets whose union equals U (i.e., covers every element of U)?

7.3.2 Direction

Vertex Cover $\leq_p$ Set Cover — again following the correct direction: reduce the already-known NP-complete problem (Vertex Cover) to the new problem (Set Cover), to help prove Set Cover is NP-hard.

7.3.3 The Core Idea (Intuitive, No Proof)

Construction (conceptual level): - Let $U = E$, the set of all *edges* of the graph G (so each element of the “universe” U is one edge of G). - For each vertex u_i in G , create a subset S_i containing **all the edges incident to u_i** (all edges touching that vertex).

Why this makes sense intuitively: “Covering all edges with vertices” (vertex cover) is directly mirrored by “covering all elements of U (the edges) using subsets S_i (each representing ‘the edges touched by vertex i ’).” Picking a vertex $\leftrightarrow$ picking its corresponding subset; an edge being “covered” by a chosen vertex $\leftrightarrow$ the corresponding element (edge) being included in the union of chosen subsets.

Concrete slide example: - Graph with vertices $u_1 \dots u_7$ and edges labeled 1–8. - $U = \{1, 2, 3, 4, 5, 6, 7, 8\}$ (the edges). - $S_1 = \{1, 2, 3\}$, $S_2 = \{1, 4\}$, $S_3 = \{3, 6\}$, $S_4 = \{4, 5, 6, 7\}$, $S_5 = \{2, 5\}$, $S_6 = \{7, 8\}$, $S_7 = \{8\}$ — each S_i lists the edge-labels incident to vertex u_i .

7.3.4 The Two Directions of the Equivalence (Conceptual)

- “ **$\exists$ a vertex cover of size $k \implies \exists$ a set cover of size k** ”: If $\{u_1, \dots, u_k\}$ is a vertex cover in G , then by construction, $S_1 \cup S_2 \cup \dots \cup S_k = U$ (because every edge in G has at least one endpoint among these vertices, i.e., every edge label appears in at least one S_i for i in the cover) — since U is defined as the set of *all* edges, this union must equal all of U .
- “ **$\exists$ a set cover of size $\leq k \implies \exists$ a vertex cover of size $\leq k$** ”: If $S_1, S_2, \dots, S_l$ ($1 \leq l \leq k$) is a set cover of U , then every edge in G is incident to one of the vertices corresponding to $S_1, \dots, S_l$ (since each edge/element being “covered” means it appears in at least one chosen S_i , i.e., is incident to that vertex) — so these $1 \leq l \leq k$ vertices form a valid vertex cover of size $\leq k$.

7.3.5 What Happens to the Instance and the Solution

- **Input transformation:** The graph’s vertex set becomes the collection of subsets (one subset per vertex); the graph’s edge set becomes

the universe U to be covered.

- **Solution translation:** A chosen collection of subsets directly corresponds (“subset S_i chosen” $\leftrightarrow$ “vertex u_i chosen”) to a vertex cover, and vice versa — no further transformation of the solution is needed beyond this direct labeling correspondence.
- **Size preservation:** The size k is preserved exactly — a vertex cover of size k corresponds to a set cover of size k (unlike the Independent Set $\leftrightarrow$ Vertex Cover case in §7.1, where the size changed to $n - k$).

7.3.6 Why This Reduction Matters / Is Useful

It shows **Set Cover is at least as hard as Vertex Cover** — i.e., Set Cover is NP-hard (since Vertex Cover is already known to be NP-complete). Combined with showing Set Cover $\in$ NP (a proposed collection of $\leq k$ subsets can be checked in polynomial time by just computing their union and comparing it to U), this establishes **Set Cover is NP-complete**.

7.3.7 Conceptual Exam Questions (No Proof Required)

● **VERY IMPORTANT | Easy** Q: In the Vertex Cover $\rightarrow$ Set Cover reduction, what does the universe U represent, and what does each subset S_i represent? A: U represents the set of all edges of the graph G . Each subset S_i represents all the edges incident to (touching) vertex u_i .

● **VERY IMPORTANT | Medium** Q: Why does a vertex cover of size k in G correspond to a set cover of size k in the constructed instance? A: A vertex cover of size k means k vertices collectively touch every edge in G . Since each vertex’s corresponding subset S_i contains exactly the edges it touches, the union of those k subsets contains every edge — i.e., equals U . So the same k vertices’ corresponding subsets form a set cover of size k .

● **IMPORTANT | Medium** Q: Does the size parameter k change during this reduction (compare to the Independent Set $\rightarrow$ Vertex Cover case)? A: No — the size is preserved exactly: a vertex cover of size k maps to a set cover of size k , and vice versa. This is different from the Independent Set $\leftrightarrow$ Vertex Cover relationship, where the size changes from k to $n - k$.

● **IMPORTANT | Medium** Q: What conclusion can we draw about the difficulty of Set Cover from this reduction? A: Since Vertex Cover (a known NP-complete problem) reduces to Set Cover in polynomial time, Set Cover is at least as hard as Vertex Cover — i.e., Set Cover is NP-hard. Together with the fact that Set Cover $\in$ NP (easily verifiable), this shows Set Cover is NP-complete.

● **IF TIME | Easy** Q: In the slide’s worked example, which subsets would you choose to form a set cover, corresponding to a vertex cover $\{u_1, u_4\}$? (Just interpret the construction — no need to verify optimality.) A: You would choose S_1 (for u_1) and S_4 (for u_4); their union $S_1 \cup S_4 = \{1,2,3\} \cup \{4,5,6,7\} = \{1,2,3,4,5,6,7\}$ — check against $U = \{1,\dots,8\}$: element 8 is missing, so $\{u_1, u_4\}$ alone would not be a valid vertex cover of this particular graph (this illustrates how to *check* set-cover validity by taking the union and comparing to U — the actual minimum cover for this graph would need to be determined by inspecting which vertices are needed to include edge 8, e.g., u_6 or u_7).

7.3.8 Common Traps for This Reduction

- Don’t mix up what plays the role of U vs. what plays the role of the subsets: **U = edges of G , subsets = per-vertex “edges touched”** — reversing this is a common slip.
- Remember the direction: **Vertex Cover $\leq_p$ Set Cover** (not the reverse) — this proves Set Cover is NP-hard by “borrowing” Vertex Cover’s known hardness.
- Unlike §7.1 (size changes to $n-k$), here the size k is preserved unchanged between the two problems — don’t apply the $n-k$ rule from the other reduction by mistake.

7.4 Summary Table: All Three Reductions at a Glance

Reduction	Direction	What’s Reused/Built	Size Relationship	Purpose
Independent Set $\leftrightarrow$ Vertex Cover	Same graph, both directions (iff)	Same graph G ; solution = complement $V-S$	$k \leftrightarrow n-k$	Shows the two problems are “mirror images” via set complement
3-SAT $\leq_p$ Independent Set	3-SAT (known hard) $\rightarrow$ Independent Set (new)	New graph built via gadgets: triangle per clause + x/x' cross-edges	k = number of clauses	Helps prove Independent Set is NP-hard (part of proving NP-completeness)
Vertex Cover $\leq_p$ Set Cover	Vertex Cover (known hard) $\rightarrow$ Set Cover (new)	New instance: U = edges, S_i = edges incident to vertex i	k stays the same ($k \leftrightarrow k$)	Helps prove Set Cover is NP-hard (part of proving NP-completeness)

7.5 General Conceptual Questions That Apply to All Three Reductions

● **VERY IMPORTANT | Easy** Q: In general, what does it mean when we say “Problem A $\leq_p$ Problem B”? A: There is a polynomial-time transformation of any instance of A into an instance of B, and a polynomial-time way to translate B’s solution/answer back into a correct solution/answer for A — so that B being efficiently solvable would make A efficiently solvable too.

● **VERY IMPORTANT | Medium** Q: For each of the three reductions in this chapter, identify which problem is the “known hard” problem and which is the “new” problem being proven hard. A: (1) Independent Set $\leftrightarrow$ Vertex Cover — this one isn’t a hardness reduction in the usual one-directional sense; it’s a structural equivalence shown both ways via complementation. (2) 3-SAT (known) $\leq_p$ Independent Set (new) — used to help show Independent Set is NP-hard. (3) Vertex Cover (known) $\leq_p$ Set Cover (new) — used to help show Set Cover is NP-hard.

● **IMPORTANT | Medium** Q: What is preserved by a correct polynomial-time reduction, in general? A: The YES/NO answer is preserved (a

YES instance of A maps to a YES instance of B, and a NO instance of A maps to a NO instance of B), and the overall time complexity relationship is preserved in the sense that a polynomial-time algorithm for B, combined with the reduction's transform/translate steps, yields a polynomial-time algorithm for A.

● **IF TIME | Medium** Q: Why is it useful to have *multiple* problems all shown NP-complete via chains of reductions (e.g., 3-SAT $\rightarrow$ Independent Set $\rightarrow$ Vertex Cover-related, Vertex Cover $\rightarrow$ Set Cover), rather than proving each one hard "from scratch"? A: Because reductions compose — once you have one problem proven NP-complete (e.g., via Cook-Levin for SAT/3-SAT), you can "bootstrap" NP-completeness proofs for many other problems just by reducing the known-hard problem to each new one, rather than directly analyzing every possible algorithm for the new problem. This builds up a large, interconnected web of NP-complete problems all shown hard relative to one foundational problem.

Topic 8: A Closing Note — HAM CYCLE and TSP

8.1 What the Slide Says

The final slide is brief but conceptually dense:

“HAM CYCLE $\rightarrow$ TSP” and “TSP $\rightarrow$ HAM CYCLE” — i.e., reductions exist **in both directions** between the Hamiltonian Cycle problem and the Traveling Salesman Problem (TSP, decision version).

“We want to solve HAM CYCLE by using TSP.” “There is a TSP [solution] $\Rightarrow$ There is a HAM CYCLE [solution].” “There is a HAM CYCLE [solution] $\Rightarrow$ There is a TSP [solution].” “Do not forget to prove that TSP is in NP.”

8.2 Simple Explanation

- **Hamiltonian Cycle (HAM CYCLE):** Does a graph G contain a cycle that visits every vertex exactly once and returns to the start?
- **TSP (decision version):** Given a weighted graph and a bound B , is there a tour visiting every vertex exactly once (a cycle) with total weight $\leq B$?

These problems are reducible to each other in **both directions**, which is why the slide shows both arrows. This is a common pattern for closely related problems: if you can efficiently solve either one, you can efficiently solve the other, so they are of **equal difficulty** (this is stronger than a one-directional $\leq_p$ — it’s a two-way equivalence).

Intuitive idea (general knowledge, consistent with the slide’s brief note, not proven in detail on the slides): HAM CYCLE is essentially “TSP where every edge has weight 1 and you ask if a tour of total weight exactly n (the number of vertices) exists” — so HAM CYCLE is a special case of TSP. Conversely, general TSP instances can be related back to HAM CYCLE-style reasoning through careful constructions (not detailed on your slides).

8.3 The Key Reminder: “Do Not Forget to Prove TSP Is in NP”

This is a **process reminder**, not a proof — it emphasizes that when doing an NP-completeness proof via a reduction (like showing TSP is NP-complete using HAM CYCLE), you must **not skip** the “membership in NP” requirement from the NP-completeness definition (Topic 6.5): the reduction alone (known-hard $\leq_p$ new problem) only proves NP-hardness; you separately need to show the new problem is *also* in NP to conclude full NP-completeness.

8.4 Likely Exam Questions

● **VERY IMPORTANT | Easy** Q: What two problems does the last slide connect, and in which direction(s)? A: Hamiltonian Cycle (HAM CYCLE) and the Traveling Salesman Problem (TSP) — reductions are shown in **both** directions (HAM CYCLE $\leq_p$ TSP and TSP $\leq_p$ HAM CYCLE).

● **VERY IMPORTANT | Medium** Q: Why does the slide remind us “do not forget to prove that TSP is in NP”? A: Because proving NP-completeness requires two separate ingredients: (1) NP-hardness, typically shown via a reduction from a known NP-complete/hard problem, and (2) membership in NP itself (a polynomial-time verifier). A reduction alone (even from a known-hard problem like HAM CYCLE) only establishes NP-hardness — it is easy to forget the separate, equally necessary step of showing the target problem (TSP) is itself in NP.

● **IMPORTANT | Medium** Q: What does it mean that reductions exist between HAM CYCLE and TSP in *both* directions, as opposed to just one? A: It means the two problems are of essentially equivalent difficulty — an efficient algorithm for either one would give you an efficient algorithm for the other. A single-direction reduction ($A \leq_p B$) only shows B is *at least* as hard as A ; showing reductions both ways establishes they are (up to polynomial factors) equally hard.

8.5 Common Mistakes

- Don’t assume a reduction from a known-hard problem is, by itself, a complete NP-completeness proof — you always also need the separate “membership in NP” check (see §6.5’s full recipe).
- Don’t assume HAM CYCLE and TSP are “the same problem” just because reductions go both ways — they remain formally distinct problems; “both ways reducible” means equally hard, not identical.

Master List: Definitions to Memorize

For each, a **formal definition** (as close as possible to the slides), a **simple meaning**, and a short **exam version** you can reproduce quickly under time pressure.

Decision Problem

Formal: A problem whose output is a single boolean value: YES or NO. **Simple meaning:** A problem you answer with just “yes” or “no.” **Exam version:** “A problem whose answer is always YES or NO.”

P

Formal: The set of decision problems that can be solved in polynomial time. **Simple meaning:** Problems with a fast, direct-solving algorithm. **Exam version:** “Decision problems solvable in polynomial ($O(n^k)$) time.”

NP

Formal: The set of decision problems with a polynomial-time verification for a “YES” answer — given an instance I and a proposed solution S , we can efficiently check whether S corresponds to a YES answer. **Simple meaning:** Problems where a YES answer, once you have a candidate solution, is fast to double-check (even if finding that solution was hard). **Exam version:** “Decision problems whose YES-instances have a certificate checkable in polynomial time.”

co-NP

Formal: The set of decision problems with a polynomial-time verification for a “NO” answer; equivalently, the set of complements of NP problems. **Simple meaning:** The “opposite” of NP — fast checking of NO answers instead of YES answers. **Exam version:** “Decision problems whose NO-instances have a certificate checkable in polynomial time.”

Complement of a Problem (X')

Formal: For a decision problem X , X' is the problem where the answer is YES exactly when X 's answer is NO, and vice versa. **Simple meaning:** Flip every answer. **Exam version:** “ X' answers YES wherever X answers NO, and NO wherever X answers YES.”

Polynomial-time Reduction ($A \leq_p B$)

Formal: A is polynomial-time reducible to B if there exist polynomial-time functions f and h such that for every instance I of A , solving $f(I)$ as an instance of B and applying h to the result gives a correct solution to I . **Simple meaning:** You can solve A by translating it into B , letting a B -solver handle it, and translating the answer back — all in polynomial extra time. **Exam version:** “ A can be transformed into B in polynomial time such that solving B solves A .”

NP-hard

Formal: Problem X is NP-hard if for all $Y \in \text{NP}$, $Y \leq_p X$. **Simple meaning:** Every NP problem can be converted into X — X is at least as hard as everything in NP. **Exam version:** “Every problem in NP reduces to X in polynomial time (X need not itself be in NP).”

NP-complete

Formal: Problem X is NP-complete if (i) $X \in \text{NP}$, and (ii) X is NP-hard. **Simple meaning:** X is both “checkable fast” (in NP) and “at least as hard as everything in NP” (NP-hard) — the hardest problems that are still in NP. **Exam version:** “ $X \in \text{NP}$ AND X is NP-hard.”

Vertex Cover (decision problem)

Formal: Given a graph G and number k , does G contain a set S of at most k vertices such that every edge has at least one endpoint in S ? **Exam version:** “ $\leq k$ vertices touching every edge.”

Independent Set (decision problem)

Formal: Given a graph G and number k , does G contain a set I of at least k pairwise non-adjacent vertices? **Exam version:** “ $\geq k$ vertices, no two adjacent.”

Set Cover (decision problem)

Formal: Given a universe U of n elements and subsets $S_1, \dots, S_m$ of U , is there a sub-collection of at most k subsets whose union is U ? **Exam version:** “ $\leq k$ subsets whose union covers everything.”

Master List: Theorems & Key Results

For each: statement, meaning, why it matters, and what to remember.

- 1. $P \subseteq NP$.** *Meaning:* Every efficiently-solvable decision problem is also efficiently-verifiable. *Why it matters:* Establishes the basic containment underlying the whole P-vs-NP picture. *Remember:* Proof idea — an efficient solver can be turned into an efficient certifier that just ignores the certificate.
- 2. P is closed under complementation ($x \in P \implies x' \in P$).** *Meaning:* If a problem is efficiently solvable, so is its “flip the answer” version. *Why it matters:* Used as the key stepping stone for $P \subseteq \text{co-NP}$ and for the $P=NP \implies NP=\text{co-NP}$ theorem. *Remember:* Proof idea — run the same algorithm, flip the boolean output.
- 3. $P \subseteq \text{co-NP}$.** *Meaning:* Every efficiently-solvable problem also has efficiently-verifiable NO-answers. *Why it matters:* Parallels $P \subseteq NP$; completes the “P sits inside both NP and co-NP” picture. *Remember:* Chain: $x \in P \implies x' \in P \implies x' \in NP \implies x \in \text{co-NP}$.
- 4. Is $P = NP$? (Open.)** *Meaning:* Unknown whether every efficiently-verifiable problem is also efficiently-solvable. *Why it matters:* The single most famous open problem in theoretical computer science; the whole theory of NP-completeness is built around this uncertainty. *Remember:* Belief: $P \neq NP$ (P is a strict subset), but **unproven**.
- 5. Is $NP = \text{co-NP}$? (Open.)** *Meaning:* Unknown whether “easy to verify YES” and “easy to verify NO” describe the same class of problems. *Why it matters:* Believed false ($NP \neq \text{co-NP}$); proving this would be an even bigger result than resolving P vs NP. *Remember:* $P = NP \implies NP = \text{co-NP}$ (so $NP \neq \text{co-NP} \implies P \neq NP$, the useful contrapositive).
- 6. Difficulty flows from A to B; efficiency flows from B to A (under $A \leq_p B$).** *Meaning:* A reduction lets you “borrow” B’s efficient algorithm (if it exists) to solve A, and lets you “push” A’s known hardness onto B. *Why it matters:* This is the entire logical engine behind proving NP-hardness/completeness. *Remember:* If $B \in P$, then $A \in P$. If A has no efficient algorithm, neither does B.
- 7. NP-completeness proof method: show $X \in NP$, plus find one known NP-complete Y with $Y \leq_p X$.** *Meaning:* You don’t need to reduce every NP problem to X — one known-hard problem suffices, by transitivity of reductions. *Why it matters:* This is the practical, standard technique used to prove new problems NP-complete (used for Independent Set via 3-SAT, and Set Cover via Vertex Cover, in this course). *Remember:* Direction! known-hard $\leq_p$ new-problem, not the reverse.
- 8. $\text{PRIME} \in P$ (AKS algorithm).** *Meaning:* Primality testing has a polynomial-time algorithm, even though it initially seemed to only obviously belong to co-NP. *Why it matters:* Famous, celebrated result illustrating that “seems hard” $\neq$ “is hard” — a caution against premature intuition. *Remember:* $\text{PRIME} \in \text{co-NP}$ was easy to see (exhibit a factor for NO); $\text{PRIME} \in P$ was the hard, celebrated result (Agrawal, Kayal, Saxena).
- 9. Independent Set $\leftrightarrow$ Vertex Cover (iff theorem).** *Statement:* S is an independent set of $G=(V,E)$ iff $V-S$ is a vertex cover of G. *Why it matters:* Directly links the difficulty (and solutions) of the two problems via simple set complementation. *Remember:* Size relationship $k \leftrightarrow n-k$.

Master List: Common Exam Traps (Consolidated)

1. **NP does NOT stand for “Not Polynomial.”** It means polynomial-time *verifiable*. $P \subseteq NP$, so plenty of “easy” problems are in NP too.
2. **$P = NP$ is not proven false** — it’s an open problem, only *believed* false.
3. **NP-hard $\neq$ NP-complete.** NP-complete requires *also* being in NP; NP-hard alone does not.
4. **Reduction direction is everything.** To prove X is NP-hard, reduce a *known* NP-complete problem **to** X (*known $\leq_p$ X*), never the reverse. This is explicitly flagged as a common student mistake on the slides.
5. **“P is closed under complement” is proven; “NP is closed under complement” is NOT known** (in fact believed false, i.e., $NP \neq co-NP$ is believed, though unproven). Don’t assume symmetry between P’s and NP’s behavior under complementation.
6. **Verifying YES vs. verifying NO are fundamentally different tasks** for many NP problems ($\exists$ certificate for YES vs. seemingly needing a $\forall$ argument for NO) — this asymmetry is *why* NP’s closure under complement is unclear.
7. **Independent Set $\leftrightarrow$ Vertex Cover size relationship is $k \leftrightarrow n-k$, NOT $k \leftrightarrow k$.** Don’t confuse this with the Vertex Cover $\rightarrow$ Set Cover reduction, where size **is** preserved ($k \leftrightarrow k$).
8. **A reduction proves NP-hardness only — you must separately prove membership in NP** to get full NP-completeness (explicitly reinforced by the “don’t forget to prove TSP is in NP” reminder on the slides).
9. **Shortest Path and MST are efficiently solvable — they are NOT NP-hard.** They’re explicitly called out as *exceptions* to the general pattern of hard search problems, not typical examples of hardness.
10. **Decision problems require a bound/threshold when converted from optimization problems** (e.g., “is the shortest path $\leq x$?”) — omitting the bound is a common error that breaks the decision-problem framing.
11. **PRIME being in co-NP does not prevent it from also being in P** — being in co-NP and being in P are not mutually exclusive; don’t assume “hard-looking” complement-verification rules out an efficient direct algorithm.

How to Write Computational Complexity Answers (Templates)

Use these structures when you see a question that “sounds abstract” and you don’t know where to start.

Template: “Define X”

1. State the formal definition, word-for-word if possible (from the Definitions master list).
2. Add one sentence of simple/intuitive meaning if time allows. *Keep it short — 1-3 sentences total for a pure definition question.*

Template: “Explain X” / “Explain why X”

1. **Restate** what X claims, in your own words.
2. **Give the reasoning/mechanism** — the “why”, using the specific logical chain from the slides (e.g., “because we can construct an algorithm/certifier that does Y”).
3. **Connect to the bigger picture** — one sentence on why this fact matters in the theory (optional, if time allows).

Template: “Differentiate X and Y”

1. State X’s definition.
2. State Y’s definition.
3. Give **one clear sentence** naming the key distinguishing condition (e.g., “NP-complete requires membership in NP; NP-hard does not”).
4. If possible, give a concrete example that is one but not the other.

Template: “Why is X important / useful?”

1. State what problem X solves or what gap X fills (e.g., “since we can’t directly prove no efficient algorithm exists, X lets us...”).
2. State the practical consequence (e.g., “so a fast algorithm for one NP-complete problem would solve all of NP”).

Template: “Explain a reduction” (proof NOT required, e.g., the three named reductions)

1. State the **two problems** involved and the **direction** of the reduction ($A \leq_p B$ — which is known-hard, which is new).
2. Describe the **core idea** of the construction in one or two sentences (what structure is built, and what it represents) — **without** reproducing the full formal proof.
3. State **what is preserved** ($YES \leftrightarrow YES$, $NO \leftrightarrow NO$) and, if relevant, how the **solution size** relates between the two problems (same k , or a transformed k like $n-k$).
4. State the **conclusion**: what this reduction tells us about relative hardness (e.g., “this shows Set Cover is at least as hard as Vertex Cover, i.e., Set Cover is NP-hard”).

Template: “Prove that $X \subseteq Y$ ” (short set-containment proofs, e.g., $P \subseteq NP$, $P \subseteq co-NP$)

1. **Start**: “Let x be an arbitrary problem in X .”
2. **Use the definition of X** to say what you’re given (e.g., a polynomial-time algorithm A for x).
3. **Construct** the required object for Y (e.g., a certifier, or a complement-solving algorithm), explicitly describing what it does.
4. **Verify** it runs in polynomial time and is correct.
5. **Conclude**: “Hence $x \in Y$. Since x was arbitrary, $X \subseteq Y$.”

Template: “What can you conclude if...?” (implication-chasing questions)

1. Identify exactly which theorem/implication applies (e.g., “if $B \in P$, and $A \leq_p B$, then...”).
2. Write out the implication chain step-by-step, citing each rule used (don’t skip steps — examiners often give partial credit per correct link in the chain).
3. State the final conclusion clearly in one sentence.

Final Last-Minute Revision Sheet

(Read this section right before walking into the exam.)

MUST MEMORIZE

- **P**: decision problems solvable in poly time.
- **NP**: decision problems with poly-time verification of YES.
- **co-NP**: decision problems with poly-time verification of NO (= complements of NP problems).
- **NP-hard**: every $Y \in \text{NP}$ satisfies $Y \leq_p X$.
- **NP-complete**: NP-hard **and** $X \in \text{NP}$.
- **$A \leq_p B$** : poly-time transform A-instance $\rightarrow$ B-instance, solve with B's algorithm, poly-time translate back.
- $P \subseteq \text{NP}$ (proven). $P \subseteq \text{co-NP}$ (proven). P closed under complement (proven).
- $P = \text{NP}$? **Open**. $\text{NP} = \text{co-NP}$? **Open**. $P = \text{NP} \implies \text{NP} = \text{co-NP}$.
- To prove NP-completeness: show $X \in \text{NP}$, then find known-hard Y with **$Y \leq_p X$** (direction matters!).
- Vertex Cover, Independent Set, Set Cover, SAT/3-SAT are all NP-complete-related problems discussed.
- Independent Set $\leftrightarrow$ Vertex Cover: S independent $\iff V - S$ is a vertex cover (size $k \leftrightarrow n - k$).
- $\text{PRIME} \in P$ (AKS); PRIME' 's NO-answers were always easy ($\in \text{co-NP}$).

MUST UNDERSTAND (don't need word-perfect recall)

- Why proving "no efficient algorithm" is intrinsically hard (vs. just failing to find one).
- Why optimization and decision problems are essentially equally hard (reduce to each other).
- Why "hard to find, easy to check" is the intuition for NP.
- Why the $\exists$ (YES-certificate) vs. $\forall$ (NO-certificate) asymmetry blocks an easy proof that NP is closed under complement.
- Why reductions must go **known-hard $\rightarrow$ new-problem**, not the reverse.
- The "big picture" Venn diagram: $P \subseteq \text{NP}$, with NP-complete sitting at the overlap of NP and NP-hard, and NP-hard extending beyond NP.

REDUCTIONS (conceptual only — no proof needed)

Reduction	Direction	Main idea	What it establishes	Don't memorize
Independent Set $\leftrightarrow$ Vertex Cover	Same graph, both ways	Complement of an independent set is a vertex cover	The two problems are structurally linked ($k \leftrightarrow n - k$)	The full formal proof text
3-SAT $\rightarrow$ Independent Set	3-SAT (known) $\rightarrow$ IS (new)	Triangle per clause (≤ 1 pick each); x/x' edges for consistency	Independent Set is NP-hard	The construction diagrams/formal correctness proof
Vertex Cover $\rightarrow$ Set Cover	VC (known) $\rightarrow$ SC (new)	$U = \text{edges}$; $S_i = \text{edges incident to vertex } i$	Set Cover is NP-hard	The formal correctness proof

COMMON TRAPS (top 5 to avoid)

1. $\text{NP} \neq$ "not polynomial" — it's about verification.
2. NP-hard $\neq$ NP-complete (membership in NP is the extra condition).
3. Reduce **known-hard $\rightarrow$ new problem**, never the reverse, when proving NP-hardness.
4. $\text{IS} \leftrightarrow \text{VC}$ size is **$k \leftrightarrow n - k$** ; $\text{VC} \rightarrow \text{SC}$ size is **$k \leftrightarrow k$** — don't mix these up.
5. A reduction alone only proves NP-hardness — always separately confirm membership in NP for full NP-completeness.

TOP EXAM QUESTIONS TO REVIEW RIGHT BEFORE THE EXAM

1. Define P, NP, co-NP, NP-hard, NP-complete (word-perfect if possible).
2. Prove $P \subseteq \text{NP}$. Prove $P \subseteq \text{co-NP}$. Prove P closed under complementation.
3. Prove $P = \text{NP} \implies \text{NP} = \text{co-NP}$.
4. Explain "difficulty flows from A to B; efficiency flows from B to A."
5. Describe the standard method (and correct direction) for proving a new problem NP-complete.
6. Explain, conceptually, what each of the three named reductions shows and why (no proofs).
7. State the Independent Set $\leftrightarrow$ Vertex Cover theorem and compute a quick size example ($k \leftrightarrow n - k$).
8. Explain why SAT is in NP but its P-membership is unknown.
9. Explain the PRIME example (co-NP vs. P) and what it teaches about intuition being wrong.
10. Explain why converting an optimization problem to a decision problem needs a bound, with the shortest-path example.

Good luck on your exam tomorrow.